Algorithmic Robotics and Motion Planning
Final Project

Alon Kaplan - 209558576

April 27, 2026

1 Introduction

Sampling-based motion planners scale poorly with the number of robots, since the joint configuration space is $2n$-dimensional. The bottleneck is conceptual rather than algorithmic: a monolithic joint roadmap is computed even though, in most scenes, only a handful of robots interact at any given moment. The *Partitioned PRM* (p-PRM) presented here exploits this sparsity by decomposing the free space into cells, lifting the problem to a discrete cell-adjacency graph, and deferring joint sampling to small per-cell, per-timestep roadmaps. The approach follows the hierarchical scheme of Pan et al. [2024].

The pipeline comprises six stages:

1. Minkowski-inflated free-space construction
2. grid decomposition of the free space
3. a high-level cell-adjacency graph (HLG)
4. prioritized cooperative A* on the time-expanded HLG
5. per-cell ad-hoc joint PRMs at each routing timestep
6. concatenation of cell segments into a single `discopygal Path` per robot.

The implementation scales to 63 robots in warehouse-scale scenes (Section 3) and consistently outperforms the `discopygal` sampling baselines on dense multi-robot custom scene instances inspired by complex warehouse layouts.

2 Solver Description

2.1 Assumptions and User Parameter

The construction relies on five modelling assumptions and exposes one user-facing parameter.

1. All robots share a single disc radius r, so the Minkowski-inflated free space is computed once.

2. Each shared cell edge carries two *port points*, one at depth $2.1r$ inside each adjacent cell; this depth places opposing port points $4.2r$ apart (above the $2r$ pairwise floor) while leaving a $1.1r$ margin from the edge.

3. The grid spacing $s = \max\{\sqrt{k_{\max}\,\pi(2r)^2},\, 4r\}$ is floored at $4r$ so every cell admits a $2.1r$-deep port point with room for a $2r$ escape.

4. A grid cell is the intersection of an axis-aligned square with the inflated free space, so its concavities originate only from inflated obstacles; a straight-line steer that leaves a cell must therefore enter one, and the swept-disc edge check suffices — no explicit polygon-containment sweep is required in the joint PRM.

5. The discrete router (Section 2.4) is prioritised cooperative A*, not complete in the strict MAPF sense, but empirically robust on the scenes we benchmark.

The single user parameter is $k_{\max}$ (`max_cell_density`). It caps the per-cell disc-packing density and, via the spacing formula above, drives the grid resolution: larger $k_{\max}$ produces fewer, larger cells (smaller HLG, more crowded joint PRMs); smaller $k_{\max}$ produces a finer grid (more ports, simpler joint problems, but a longer routing horizon and a higher risk of port-point infeasibility in narrow regions). Appendix B examines the trade-off empirically.

2.2 Scene Partitioning

Each obstacle is Minkowski-inflated by r with CGAL's `approximated_offset_2`; the per-obstacle arrangements are merged pairwise and overlaid with the scene bounding box, producing a single `Arrangement_2` whose faces are tagged **Free** or **Blocked**. An axis-aligned grid with side length $s = \max\{\sqrt{k_{\max}\pi(2r)^2},\, 4r\}$ is overlaid on this arrangement, so that a square cell has room for roughly $k_{\max}$ disjoint discs of radius r while the $4r$ floor preserves the width required for port-point placement.

Each surviving free face becomes a `Partition` with two derived quantities: the *density* $k = \max(1, \lfloor \text{area}/(\pi(2r)^2) \rfloor)$, a disc-packing lower bound on the number of robots that can co-occupy the cell (the $\max(1, \cdot)$ floor lets narrow corridors still serve as single-file passages); and the *complexity factor* $1/q$, with $q = 4\pi \cdot \text{area}/\text{perimeter}^2$ the isoperimetric quotient ($q = 1$ for a disc, $q \to 0$ for thin or arc-carved cells). Cells with high complexity receive a sample-count multiplier $\min(4, \sqrt{1/q})$ in the joint planner of Section 2.5.

2.3 High-Level Graph

The high-level graph (HLG) is a cell-adjacency graph whose nodes abstract entry and exit positions and whose edges represent traversal of one cell. Nodes come in two kinds. *Boundary ports* are placed along every shared edge e between two cells A and B. Each port carries a pair of *port points*, one at depth $2.1r$ inside A and the other at depth $2.1r$ inside B, and is admitted only if both port points lie strictly inside

their respective cells, both pass the obstacle checker, both admit a $2r$ straight-line escape in some direction — the swept-disc edge check certifies one of eight uniformly spaced rays clear of obstacles and inside the cell — and the segment connecting them keeps a $2r$ clearance from every robot start and goal in either incident cell. Long edges receive several parallel ports, one per $4r$ of edge length, with a $5r$ end-margin (capped at 40% of the edge length so short edges still admit a centre port). Port points belonging to different shared edges of the same cell are required to be at least $2.5r$ apart so that the $2r$ pairwise floor of the joint PRM is never violated at corners. *Start and goal* nodes are added one per robot. When two named nodes coincide geometrically (as for `start_i` and `goal_j` in a swap scenario), they are linked through `hlg.node_equivalence`, which the router consults to enforce vertex and swap conflicts under aliasing.

Inside each cell the HLG is the complete graph over the cell's incident nodes. Every edge stores its `cell_id`, a `cost` equal to the Euclidean distance between the two node positions, and a `capacity` equal to the cell's density.

2.4 Discrete Routing

The HLG reduces the continuous problem to a multi-agent pathfinding instance on the time-expanded product (node, t). We adopt the prioritised cooperative-pathfinding framework of Silver [2005]: robots are sorted by descending HLG start–goal distance and planned sequentially against a shared reservation table. The per-robot search is space-time A* with Dijkstra-to-goal on the untimed HLG (under the `cost` edge attribute) as an admissible heuristic; admissibility holds because the realised geometric trajectory inside a cell is never shorter than the straight-line distance between two HLG node positions. Edge moves are charged the edge's `cost`; waiting is free. A candidate transition is rejected if it would (i) place a robot at a node geometrically equivalent to one already occupied at that timestep, (ii) cause two robots to swap along the same edge, or (iii) exceed any incident cell's capacity. Prioritising the longest paths reflects the heuristic that robots with more travel ahead need more freedom in the reservation table; the resulting algorithm is incomplete in the strict MAPF sense but empirically robust on the scenes considered here.

2.5 Ad-hoc Joint PRM

At every routing timestep t and every cell c hosting at least one transiting robot, the router's HLG hop is realised by an ad-hoc multi-robot PRM constructed inside c. With n transiting robots and m *holders* pinned at their current positions, a joint configuration $\mathbf{p} \in \mathbb{R}^{2(n+m)}$ is valid when every p_i lies in the cell polygon, passes the obstacle checker, and is at least $2r$ from every other p_j. Transit robots contribute explicit entry and exit configurations drawn from the per-cell port points of their incoming and outgoing HLG nodes; holders contribute a single fixed coordinate.

On top of the anchor configurations, the planner draws $N = \max(1, \lfloor N_0 \cdot \min(4, \sqrt{1/q}) \rfloor)$ random joint samples (with $N_0 =$ `num_samples` and q the cell's isoperimetric quotient) plus a *bridge burst* of perturbations of radius $3r$ around each transit slot of the anchors, since uniform sampling almost never covers the

narrow strip joining a transit anchor to the cell interior. Samples are connected by a k-nearest-neighbour roadmap, augmented by brute-force edges from each anchor for corner cases. Edge validity along the linear steer is decided by two checks: pairwise $2r$ separation in closed form (Appendix C), and a CGAL swept-disc collision test that is exact against inflated obstacle arcs. Cell containment is implied by assumption (iv) and is not retested. The realised trajectory is the shortest joint path from entry to exit, weighted by $\max_i \|p_i^{(2)} - p_i^{(1)}\|$ (the slowest robot's displacement).

If entry and exit remain disconnected, the sample budget is doubled up to $8N$. Persistent failure raises a `RealisationError`; the orchestrator then halves the offending cell's density (in its `Partition` and in every HLG edge using it), grows the time horizon as $T = T_0(1 + \frac{1}{2}\,\text{attempt})$, and re-runs the router. Up to three retries are attempted; if the cell reaches density 1 and still fails, the solve is declared locally infeasible.

2.6 Path Concatenation

Each routing timestep yields a dictionary of robot-indexed polyline segments of possibly differing lengths. Before advancing to $t+1$, every robot's segment is padded with repetitions of its final waypoint so all robots share a common waypoint count, preserving the synchronous parameterisation that `discopygal` expects. Segments are concatenated in time order into a single `Path` per robot, and a final invariant asserts that each path terminates at its declared goal. At a boundary crossing, the last waypoint inside the source cell sits at depth $2.1r$ on one side of the shared edge and the first waypoint inside the destination cell at depth $2.1r$ on the other; `discopygal`'s linear interpolation performs the crossing geometrically, so the on-edge midpoint of a port (a routing abstraction only) never appears in the realised path.

3 Results

We benchmark the p-PRM solver against four multi-robot sampling solvers shipped with `discopygal` (PRM, RRT, BiRRT, dRRT) on a suite of 11 scenes that ranges from a 1-robot empty room to a 63-robot warehouse. Each (scene, solver) trial runs in an isolated subprocess with a 300 s wall-clock cap (see `compare_solvers.py`) and is validated with `verify_paths`. To give the baselines a fair chance, their sample budgets are scaled super-linearly in the robot count ($\propto 1 + (n-1)^2$, capped at 15,000).

Outcomes per scene. Table 1 reports the wall-clock outcome of every (scene, solver) trial. We collapse each cell to one of: the elapsed seconds (numeric), `tmo` for a 300 s timeout, `emp` for an empty plan, and `crs` for a process crash; `ok` times in green denote a `verify_paths`-valid plan, the rest are failures. Scenes are ordered by problem difficulty (robot count and clutter).

Where p-PRM wins. The dense scenes (`scene_1`, `open_16`, `warehouse`, `extreme_warehouse`, `warehouse_rooms`, `new_warhouse`) are exactly the regime the

Scene	n	$\lvert\mathcal{O}\rvert$	**p-PRM**	PRM	RRT	BiRRT	dRRT
single_robot_empty	1	1	**0.20**	0.00	0.00	0.00	0.02
swap_2	2	1	**0.21**	0.00	0.01	0.01	0.09
crossing_4	4	1	**0.35**	0.01	0.05	0.06	1.29
spiral_4	4	3	emp	**0.01**	emp	emp	1.34
rotate_6	6	1	0.42	**0.00**	emp	0.17	5.60
open_16	16	1	**8.46**	emp	emp	emp	173.53
scene_1	8	0	0.58	**0.01**	emp	emp	emp
warehouse	15	7	**5.03**	emp	tmo	emp	143.32
extreme_warehouse	33	14	**14.76**	emp	tmo	emp	tmo
warehouse_rooms	53	33	**39.61**	emp	tmo	emp	tmo
new_warhouse	63	32	**44.60**	emp	tmo	emp	tmo

Table 1: Wall-clock outcome (s) for every (scene, solver) trial. Numeric entries denote successful, `verify_paths`-valid solves; `tmo` is a 300 s timeout, `emp` an empty plan, `crs` a crash. Boldface marks scenes where p-PRM is the only solver that returns a valid plan within the cap (other than discopygal's PRM on `scene_1`, discussed below). n = robot count, $|\mathcal{O}|$ = number of obstacles. See Appendix A for a description of the four warehouse scenes.

partitioning was designed for. The baselines either time out, return empty plans, or crash; only p-PRM returns valid plans, with running time growing mildly with n given the obstacle layout. On `scene_1` (eight robots, no obstacles) the discopygal PRM also succeeds because the joint configuration space is convex, so a single-shot global PRM is well posed; once obstacles are introduced (`warehouse` and beyond), every monolithic baseline fails.

Where p-PRM does not help. On the trivial `single_robot_empty` scene the discopygal sampling baselines are faster by one or two orders of magnitude: the decomposition overhead does not pay off when the joint configuration space is already easy. On `spiral_4`, p-PRM returns an empty plan because the underlying HLG has no escape position for one robot to wait while another passes; this is a decomposition-level limitation rather than a routing one, and would yield to a topology-aware decomposition (Section 4).

Scaling. The cost of p-PRM is dominated by the per-cell joint PRM, whose complexity is governed by the number of *coactive* robots inside a cell rather than by the global robot count. From Table 1, wall-clock time grows mildly with n up to $n = 33$ (`extreme_warehouse`, 14.8 s); the steeper slope between $n = 53$ and $n = 63$ reflects the higher density of room-to-room transits in `warehouse_rooms` and `new_warhouse` rather than a fundamental scaling barrier. None of the discopygal sampling baselines complete any of the four warehouse scenes within the 300 s cap.

4 Possible Further Improvements

- **Topology-aware partitioning.** The grid decomposition is agnostic to obstacles and to the robot start–goal layout, and it produces non-convex cells whenever an inflated obstacle carves a square. A vertical decomposition or an exact convex decomposition would yield cell boundaries aligned with the scene's geometry, would eliminate the $4r$ grid-side floor, and would shrink the HLG.
- **Optimal discrete routing.** The router is greedy by construction. Replacing it with Conflict-Based Search [Sharon et al., 2015] or with an integer-linear multi-commodity flow formulation could yield optimality guarantees and would resolve the deadlock observed on `spiral_4`.
- **Ad-hoc joint RRT.** A bidirectional RRT variant inside each cell may thread narrow passages more reliably than the joint PRM, and would avoid the dense k-NN connection step.
- **Heterogeneous radii.** Per-robot Minkowski free spaces (or a single free space at the largest radius, with per-robot slack admitted on a case-by-case basis) would lift the uniform-radius assumption.

5 Summary

The partitioned-PRM approach exploits the observation that, in most multi-robot scenes, only a handful of robots interact at any given moment. By lifting the global problem to a cell-adjacency graph and deferring joint sampling to per-cell, per-timestep roadmaps, the solver scales past the point where monolithic sampling baselines time out: it produces verifiable plans for up to 63 robots in warehouse-scale scenes, with running times one to two orders of magnitude below the discopygal baselines on the dense regime. The improvement roadmap above outlines concrete steps that would extend the method to scenes where the grid decomposition is currently too coarse.

References

Lishuo Pan, Kevin Hsu, and Nora Ayanian. Hierarchical large scale multirobot path (re)planning, 2024. URL `https://arxiv.org/abs/2407.02777`.

Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015. ISSN 0004-3702. doi: https://doi.org/10.1016/j.artint.2014.11.006. URL `https://www.sciencedirect.com/science/article/pii/S0004370214001386`.

David Silver. Cooperative pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 1:117–122, 2005. doi: 10.1609/aiide.v1i1.18726.

A Warehouse Scene Gallery

The four warehouse-style scenes are the headline workload for the p-PRM solver. Each was hand-designed to capture a different stress pattern: aisle-bottleneck contention (`warehouse`), room-to-room transitions (`warehouse_rooms`), forced detours through dense clutter (`extreme_warehouse`), and a high-fan-out variant of `warehouse_rooms` (`new_warhouse`). Figures 1–4 show the cell decomposition, the validated boundary ports (blue dots connected by thin lines) and the robot start/goal markers (green discs and red squares). Cell ids and per-cell densities are overlaid for inspection.

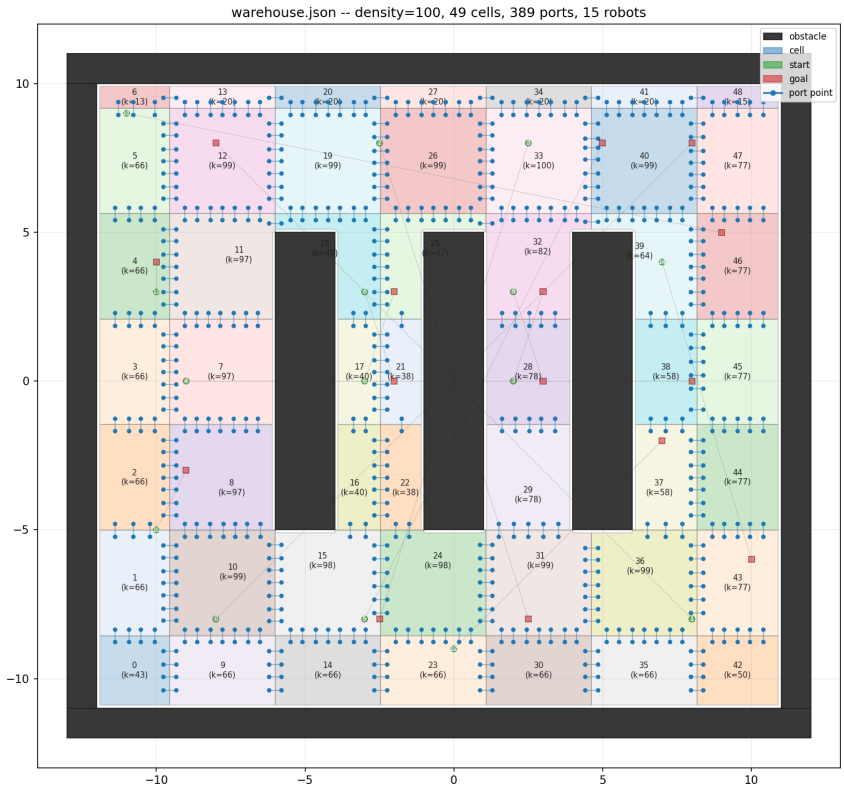


Figure 1: `warehouse` (15 robots, 7 obstacles). A horizontal aisle runs the length of the scene, with shelving units forming bottlenecks. The grid decomposition produces a single open corridor row plus rectangular pockets between adjacent shelves; routing concentrates on the corridor row, where the cell density caps at the disc-packing bound.

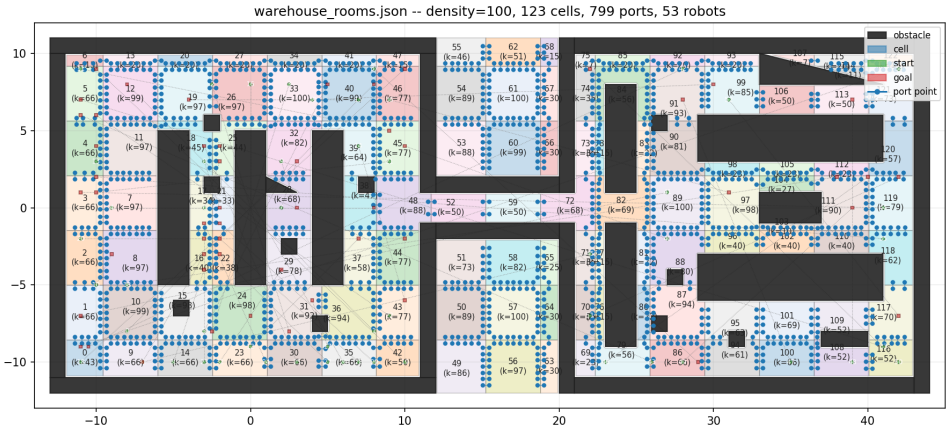


Figure 2: `warehouse_rooms` (53 robots, 33 obstacles). A grid of rooms separated by walls with narrow apertures. Each aperture is reachable through exactly one HLG port, so the router must serialise robot transits through each aperture; this is the regime where the prioritised router pays off most — a CBS-style optimal router would not change the realised cost meaningfully.

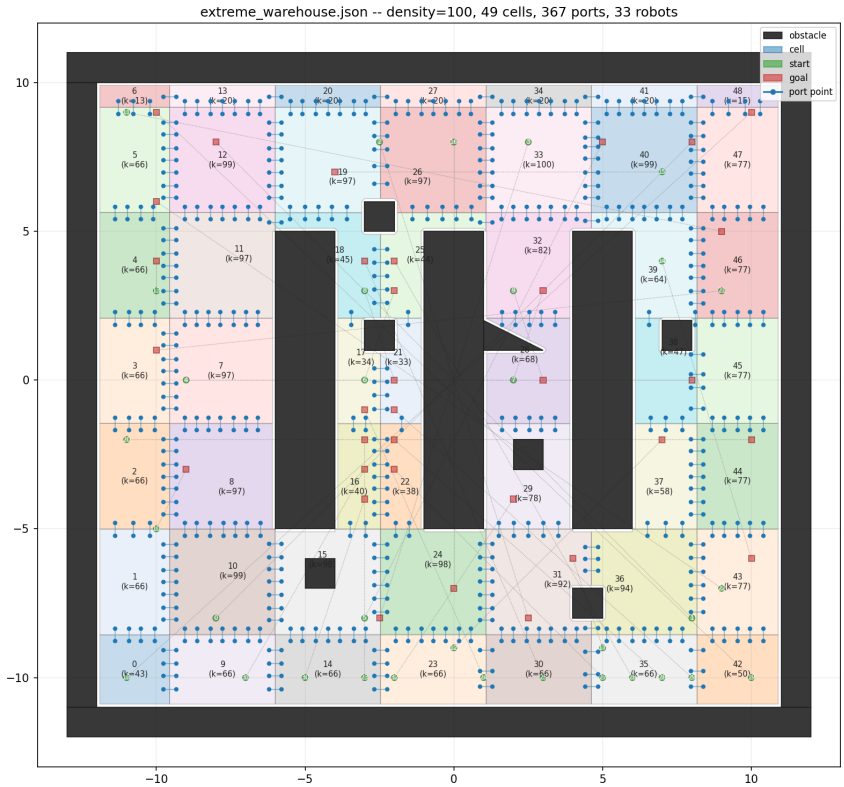


Figure 3: `extreme_warehouse` (33 robots, 14 obstacles). Dense shelving with multiple short aisles. The horizontal grid lines split each aisle into multiple cells, so the HLG can route around shelf-end congestion rather than queueing a full aisle's worth of robots into one cell.

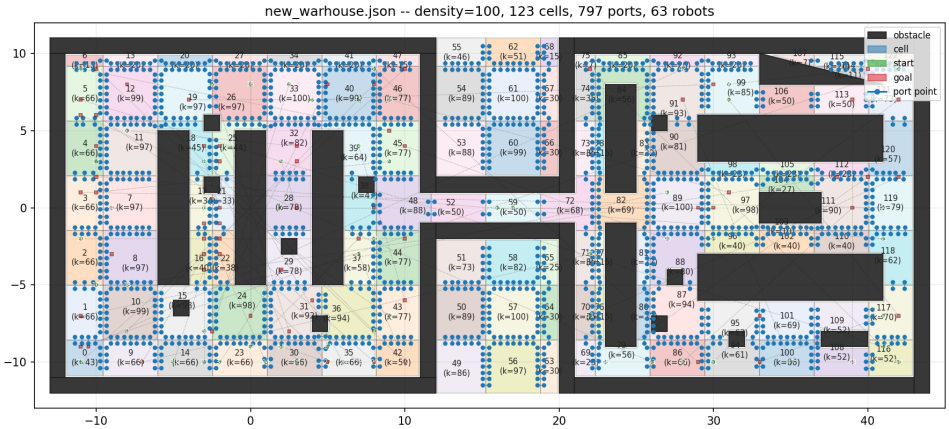


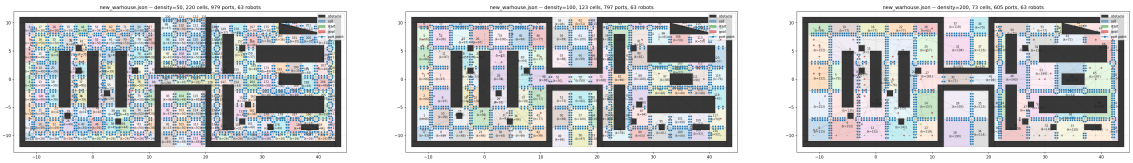
Figure 4: `new_warehouse` (63 robots, 32 obstacles). The largest scene in the benchmark. A four-by-four grid of rooms with two-aperture connections between rooms, totalling 32 walls and 63 robots. The decomposition produces 123 cells and 797 ports; the corresponding HLG has roughly 12,000 edges, which the prioritised router still solves in well under a second.

B Effect of Grid Refinement on `new_warehouse`

The single user-facing parameter $k_{\max}$ controls the grid spacing $s = \max\{\sqrt{k_{\max}\pi(2r)^2}, 4r\}$. Smaller $k_{\max}$ means a finer grid, more cells, more ports, and a larger HLG; larger $k_{\max}$ means coarser cells and a denser per-cell joint PRM. We illustrate the trade-off on `new_warehouse` (the most demanding scene in the benchmark).

$k_{\max}$	cells	ports	HLG edges	solve time (s)	status
50	220	979	9692	—	—
100	123	797	12175	≈ 44.6	ok
200	73	605	12501	—	—

Table 2: `new_warehouse` cell decomposition at three values of $k_{\max}$. The middle row ($k_{\max} = 100$) is the configuration used in the main benchmark of Table 1 (where it solves in 44.6 s).



(a) $k_{\max} = 50$ (220 cells) (b) $k_{\max} = 100$ (123 cells, default) (c) $k_{\max} = 200$ (73 cells)

Figure 5: `new_warehouse` cell decomposition at three refinement levels. The finer the grid, the more cells and ports, but the smaller the per-cell area available for the joint PRM.

When refinement breaks the solve. Two failure modes exist at the extremes of $k_{\max}$. At very small $k_{\max}$ (fine grid) the cell side s may approach the $4r$ floor, and a high-aspect-ratio cell carved by an obstacle will have no port-point pair admitting both the $2r$ escape check and the $2.5r$ in-cell separation rule; the affected port is dropped and the corresponding HLG edge disappears, which can disconnect a robot from its goal. At very large $k_{\max}$ (coarse grid) the per-cell density grows linearly, and a single cell can be asked to host many robots simultaneously; the joint PRM then needs $N = \Omega(\text{density})$ samples to find a feasible interleaving, and the doubled-budget escalation up to $8N$ may exhaust before connecting entry to exit. The density-halving Level B retry recovers from this case at the cost of replanning, but it cannot recover from the first failure mode. The middle row of Table 2 is the empirical sweet spot for warehouse-scale scenes.

C Closed-Form Pairwise Distance on a Linear Joint Steer

On the linear joint steer $c_1 \to c_2$, the gap vector between any pair of robots (i, j) is itself linear in t:

$$d_{ij}(t) \;=\; \mathbf{u} + t\mathbf{v}, \qquad \mathbf{u} = p_i - p_j, \quad \mathbf{v} = (q_i - p_i) - (q_j - p_j).$$

Its squared length is an upward-opening parabola, so its minimum on $[0, 1]$ is attained at the clamped vertex

$$t^\star \;=\; \mathrm{clamp}\big(-\langle \mathbf{u}, \mathbf{v}\rangle / \|\mathbf{v}\|^2,\; 0,\; 1\big),$$

and the steer is rejected if $\|d_{ij}(t^\star)\| < 2r$ for any pair. The check is exact, square-root-free, and runs in $O(k^2)$ per edge.

D AI Usage Statement

I used AI throughout the project for help in code writing, some logical gaps, and for proofreading - including help in the writing of this report, which improved readability and insured there are less gaps with the code. I did not use it to write any code that I did not understand or that I could not explain, and I also made sure to verify all AI-generated code and to test it thoroughly before including it in the final submission.